Week 3: pandas, random numbers, seaborn

Python syntax and code patterns. Keep it next to you while you work.

Opening a table

```
trials = pd.read_csv(ROOT / "data" / "ibl_2afc.csv.gz")
trials.head()                # first five rows
trials.shape                 # (rows, columns)
trials.dtypes                # kind of value per column
len(trials)                  # number of rows
trials["rt_s"].describe().round(3).to_string()
```

Columns

```
trials["rt_s"]               # a Series
trials[["rt_s", "correct"]]  # a smaller DataFrame
trials["fast"] = trials["rt_s"] < 0.5     # a new column
trials["rt_ms"] = trials["rt_s"] * 1000
trials.columns               # all the names
```

Rows

```
trials.iloc[0]               # by position
trials.iloc[0:5]
trials.loc[0, "rt_s"]        # by label, rows then columns
trials.loc[mask, "rt_s"]     # a column, only those rows
```

Label and position coincide on a freshly loaded table and stop coinciding the moment you filter.

Filtering

```
fast = trials[trials["rt_s"] < 0.5]

one = trials[(trials["subject_id"] == "CSHL051")
             & (trials["phase"] == "trained")]

trials[trials["rt_s"].between(0.08, 10.0)]
trials[trials["cohort"] == "CSHL-WT"]
trials[~trials["correct"].isna()]
```

&	and, each condition in its own brackets
|	or
~	not

Counting

```
(trials["rt_s"] < 0).sum()   # how many satisfy it
trials["cohort"].value_counts()
trials["subject_id"].unique()
len(trials["subject_id"].unique())
trials["rt_s"].isna().sum()  # how many are missing
trials.dropna(subset=["rt_s"])
```

Joining two tables

```
merged = trials.merge(mice, on="subject_id", how="left")
```

One row per trial, with the columns of that mouse attached to each of its trials.

Grouping

```
trials.groupby("cohort")["correct"].mean()

by_phase = trials.groupby("phase")["correct"].agg(
    n="size", accuracy="mean")
by_phase.reset_index()       # the group back as a column

per_mouse = (trials
    .groupby(["cohort", "subject_id"])["rt_s"]
    .median()
    .reset_index())
```

Out of pandas, into NumPy

```
values = per_mouse["rt_s"].to_numpy()
labs = mice["lab"].tolist()
per_mouse.sort_values("rt_s")
df.head().to_string(index=False)   # printing tidily
```

Random numbers

```
rng = np.random.default_rng(0)     # 0 is the seed
rng.normal(loc=0, scale=1, size=100)
rng.uniform(0, 1, size=100)
rng.binomial(n=1, p=0.7, size=100)
rng.poisson(lam=3, size=100)
rng.choice(values, size=20, replace=True)
```

The same seed gives the same numbers every run.

Summary numbers

```
x.mean(), x.median()
np.std(x, ddof=1), np.var(x, ddof=1)    # always ddof=1
np.percentile(x, [25, 50, 75])
sem = np.std(x, ddof=1) / np.sqrt(len(x))
round(float(value), 3)
```

Figures with seaborn

```
fig, ax = plt.subplots(figsize=(6, 4))
sns.kdeplot(x=values, ax=ax, fill=True)
sns.boxplot(data=df, x="cohort", y="rt_s", ax=ax,
            order=COHORT_ORDER, width=0.5, fliersize=0)
sns.stripplot(data=df, x="cohort", y="rt_s", ax=ax,
              color="black", size=5)
sns.violinplot(data=df, x="cohort", y="rt_s", ax=ax)
ax.set_ylabel("median RT per mouse (s)")
```

Everything seaborn draws is a Matplotlib figure, so `ax.set_xlabel` still works.

Building a table yourself

```
pd.DataFrame({"n": sizes, "power": curve})
pd.Series(values, name="rt_s")
df.round(3).to_string(index=False)     # print it tidily
```

Error bars per group

```
summary = (per_mouse.groupby("cohort")["rt_s"]
           .agg(mean="mean", sd="std", n="size")
           .reset_index())
summary["sem"] = summary["sd"] / np.sqrt(summary["n"])

ax.errorbar(summary["cohort"], summary["mean"],
            yerr=summary["sem"], fmt="o", capsize=4)
ax.set_xticks(range(len(labels)))
ax.set_xticklabels(labels, rotation=45)
```

Pattern: repeat the experiment

```
def sampling_distribution(population, n, repeats, rng):
    means = []
    for _ in range(repeats):
        means.append(rng.choice(population, size=n).mean())
    return np.array(means)

for n in [5, 20, 80, 320]:
    means = sampling_distribution(pop, n, 2000, rng)
    print(f"{n:>5} {means.std(ddof=1):>10.4f}")
```